

"Spring Boot turns our microservice design into a runnable application."

MODULE 3

Spring Boot Microservices Development

From service design to a running Java microservice.

Spring Boot
architecture

Application
lifecycle

Auto-
configuration

REST API
development

Dependency
Injection

Configuration &
profiles

Build and run

LEARNING OBJECTIVES

Architecture, lifecycle, auto-configuration, REST, DI, profiles and running services.



BY THE END OF THIS MODULE, YOU CAN

1. Explain Spring Boot architecture
2. Describe application startup
3. Understand auto-configuration
4. Manage dependencies
5. Build REST APIs
6. Apply Dependency Injection
7. Externalize configuration
8. Use Spring profiles
9. Build and run a microservice



THREE SECTIONS

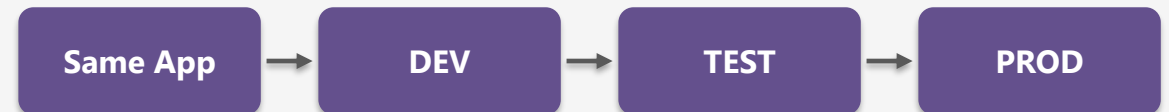
1 Understand what Spring Boot does



2 Build a real REST API



3 Make it portable across environments



KEY TAKEAWAY Understand what Spring Boot does, build a real API, then make it run in any environment.

FROM MODULE 2 TO MODULE 3

The design is ready — this module turns it into a running Spring Boot service.

MODULE 2 GAVE US

- ✓ Service boundaries from business capabilities
- ✓ Resource-oriented REST contracts
- ✓ DTOs, validation and one error format
- ✓ Stateless, replaceable instances
- ✓ A design blueprint for Order Service

NOW Build it — Order Service in the slides, Account Service in Lab 1.



OUR PATH THROUGH MODULE 3

- 1 Generate the project and see how it starts
- 2 Auto-configuration and starters
- 3 Controller, DTOs and validation
- 4 Errors, service and repository code
- 5 Wire it with dependency injection
- 6 Externalize config and use profiles
- 7 Build, run and troubleshoot one JAR

Part 1

Part 1

Part 2

Part 2

Part 2

Part 3

Part 3

KEY TAKEAWAY Module 2 decided what to build; Module 3 turns each decision into Spring Boot code.

PART 1: WHAT SPRING BOOT DOES FOR US

Before writing endpoints, understand the framework underneath them.

Part 1
What Spring Boot does

Part 2
Build the Order API

Part 3
Portable and running

SO FAR

- ✓ Service boundaries come from business capabilities
- ✓ REST contracts use resources, methods and DTOs
- ✓ Instances stay stateless; configuration lives outside

NEXT QUESTION

What does Spring Boot actually do when we build and start a service?

IN THIS PART

- 1 Why Spring Boot, and how it relates to Spring
- 2 Layered architecture and creating the project
- 3 The entry point, `@SpringBootApplication` and startup
- 4 Auto-configuration, in a picture and in code
- 5 Starters and dependency management
- 6 Exercise: choose the starters

WHY SPRING BOOT FOR MICROSERVICES?

Why Spring Boot is so common for Java microservices.



LESS SETUP, FASTER START

- ✓ Sensible defaults you can override
- ✓ Minutes from idea to running API
- ✓ Same model locally and on OpenShift

One command to run

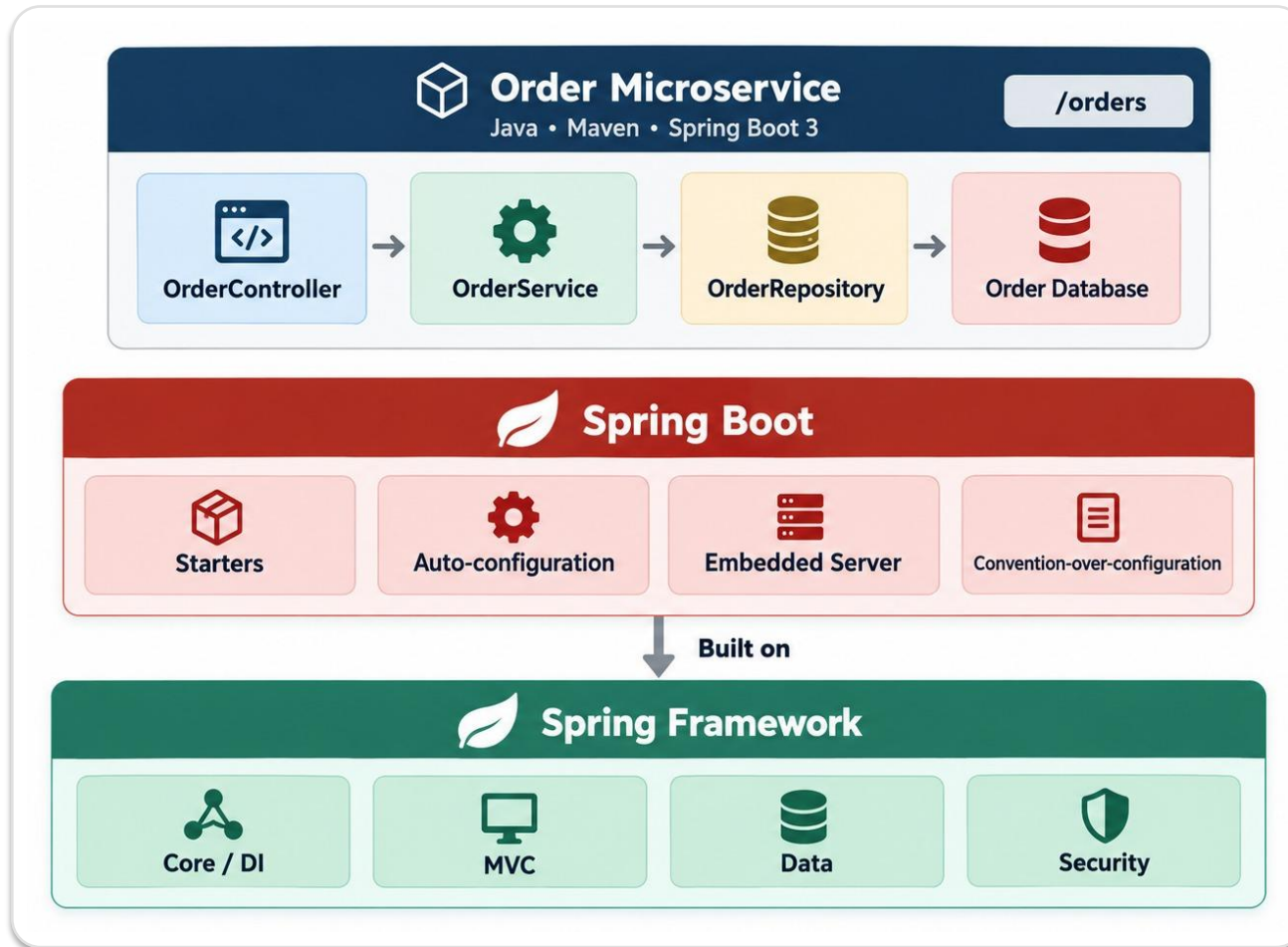
```
java -jar order-service.jar
```

WHY IT FITS Each service builds, runs and deploys on its own.

KEY TAKEAWAY Spring Boot's defaults and embedded server make independent services quick to build and run.

SPRING FRAMEWORK VS. SPRING BOOT

What Spring Boot adds on top of the Spring Framework.



MYTH Spring Boot replaced Spring. It builds on it.

Spring

Provides the capabilities: DI, web, data access, transactions, security.

Spring Boot

Provides conventions and automation to assemble them quickly.

SAME CORE, MORE MODULES

Spring Data

Spring Security

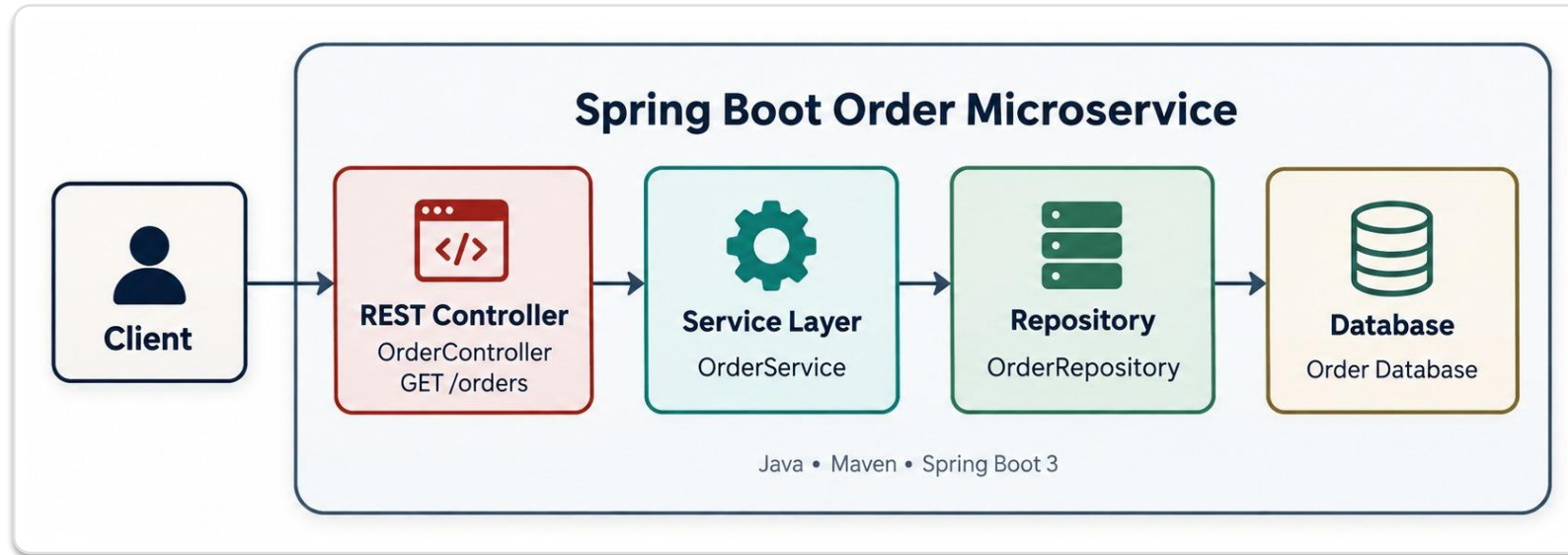
Spring Cloud

Spring Kafka

KEY TAKEAWAY Spring Boot didn't replace Spring — it makes Spring's capabilities easier to configure, package and run.

SPRING BOOT LAYERED ARCHITECTURE

Controller → Service → Repository → Database, each with one job.



Controller

HTTP only: map the URL, read input, return a status. GET `/orders/1001`

Service

Business rules and workflow: `createOrder()`, `cancelOrder()`, `calculateTotal()`

Repository

Persistence access: `save()`, `findById()`, `findAll()`

COMMON MISTAKES

- ✗ Business logic in controllers
- ✗ Entities returned from APIs
- ✗ SQL next to HTTP code

KEY TAKEAWAY Keep responsibilities separate — controllers shouldn't mix business logic and database access.

CREATING THE PROJECT WITH SPRING INITIALIZR

start.spring.io generates a ready-to-run project with the dependencies you pick.

Setting	Order Service
Project	Maven
Language	Java
Spring Boot	Current stable 3.x
Group	com.example
Artifact	order-service
Packaging	Jar
Java	17 or 21

DEPENDENCIES TO SELECT

Spring Web	Validation	Spring Data JPA
PostgreSQL Driver	Actuator	DevTools (dev only)

Or from the command line

```
curl https://start.spring.io/starter.zip \
  -d dependencies=web,validation,data-jpa,postgresql,actuator \
  -d javaVersion=21 -d artifactId=order-service \
  -o order-service.zip
```

What you get
pom.xml, the main class, application.properties, a first test and the Maven wrapper (mvnw).

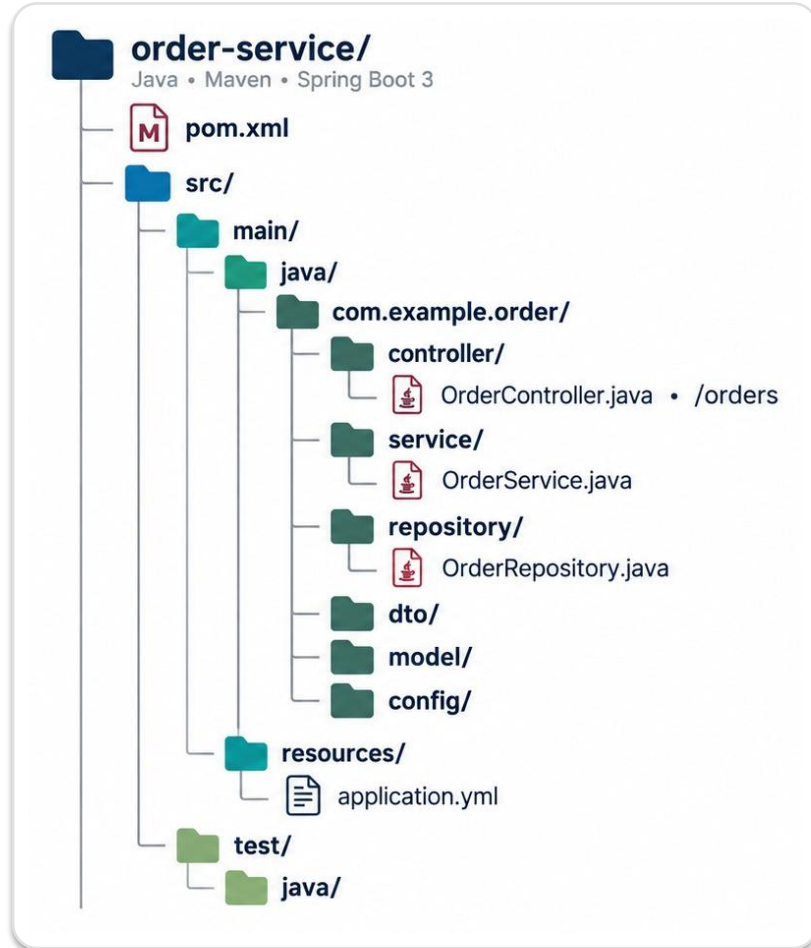
RULE Select only what this service needs — add more when a requirement appears.

KEY TAKEAWAY Spring Initializr gives every service the same clean start — with only the dependencies it needs.



ANATOMY OF A SPRING BOOT PROJECT

Where code, configuration and tests live.



pom.xml

The build file: parent version, starters and plugins. Gradle projects use build.gradle instead.

src/main/resources

application.yml and profile files — configuration stays out of Java code.

src/test/java

Tests mirror the main packages, so OrderServiceTest sits beside OrderService.

GROWING SERVICE? Group by feature — orders/, returns/ — each with its own layers. Packages alone don't create boundaries.

KEY TAKEAWAY Know where code, configuration and tests live — then give each package a clear responsibility.

THE APPLICATION ENTRY POINT

main() starts Java; SpringApplication.run() starts Spring.

OrderApplication.java

```
package com.example.order;

@SpringBootApplication
public class OrderApplication {

    public static void main(String[] args) {
        SpringApplication.run(OrderApplication.class, args);
    }
}
```

✓ What the console shows

```
:: Spring Boot :: (v3.5.0)
Starting OrderApplication using Java 21
No active profile set, falling back to "default"
Tomcat initialized with port 8080 (http)
Started OrderApplication in 2.41 seconds
```

main()

A plain Java entry point. The JVM starts here.

SpringApplication.run()

Prepares the environment, creates the ApplicationContext, wires the beans and starts Tomcat.

The package line

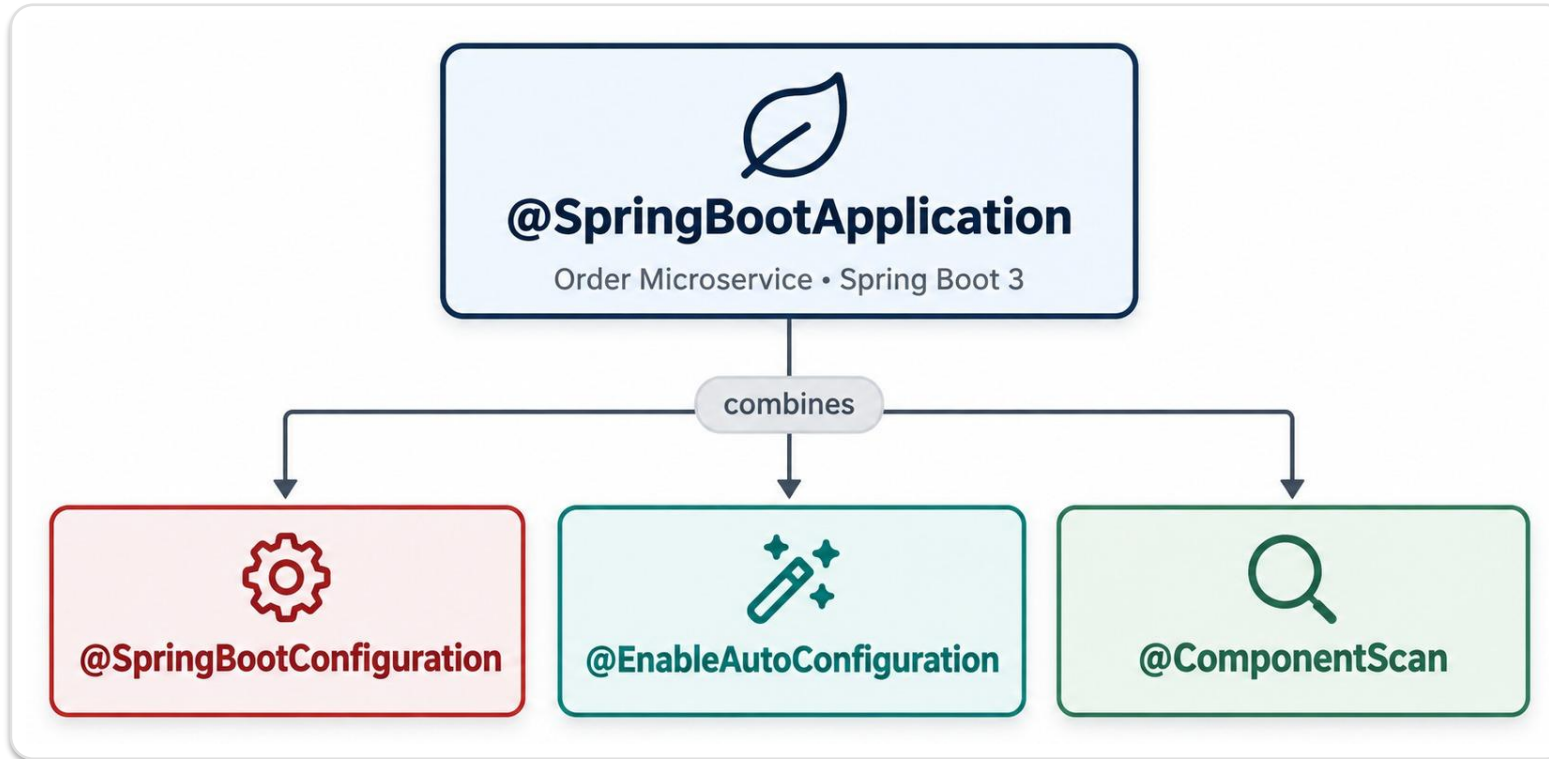
com.example.order is where component scanning starts — see the next slide.

KEEP IT EMPTY No business logic in the main class.

KEY TAKEAWAY main() hands control to SpringApplication.run(), which boots the whole application.

UNDERSTANDING @SPRINGBOOTAPPLICATION

Three annotations in one — and what each one does.



@SpringBootConfiguration

This class can declare @Bean methods.

@EnableAutoConfiguration

Boot configures infrastructure from what's on the classpath.

@ComponentScan

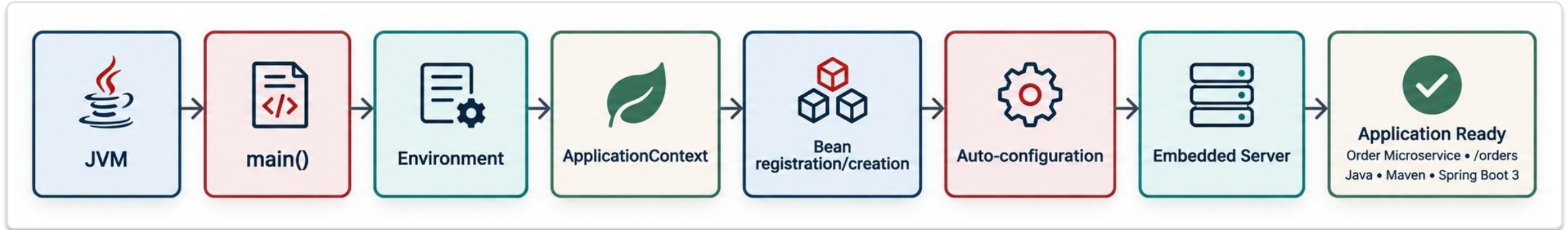
Finds components in this package and every package below.

PACKAGE MATTERS `com.example.order.web` is scanned; `com.example.shared` is not.

KEY TAKEAWAY Component scanning starts from the main class — so its package location matters.

SPRING BOOT STARTUP LIFECYCLE

JVM → environment → ApplicationContext → beans → server → ready.



✓ The line you want to see

Started OrderApplication in 2.41 seconds

Why learn it

It's your map when a service won't start — Part 3 uses it to troubleshoot.

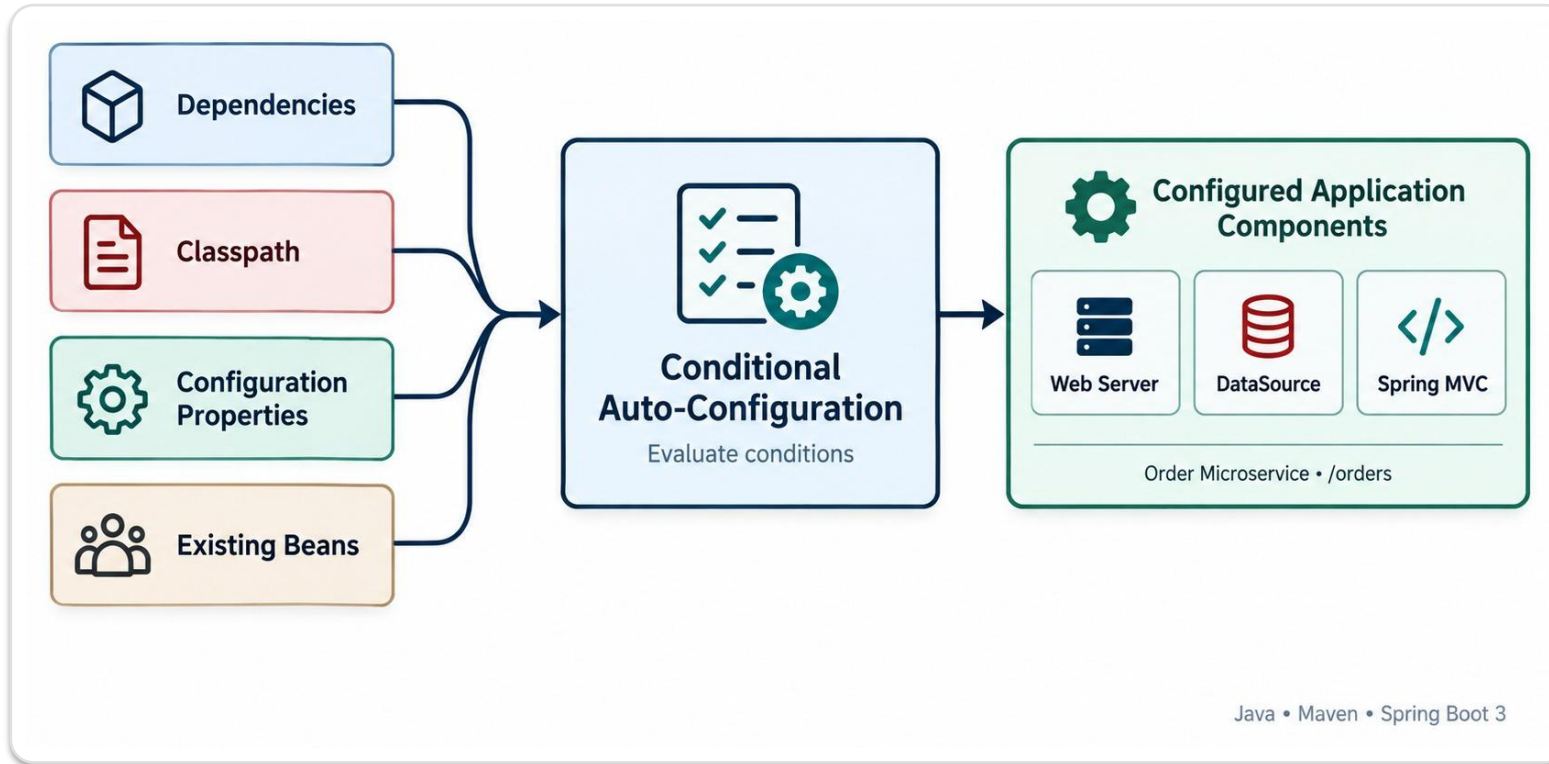
Shutdown

The context closes and releases connections and threads. `server.shutdown=graceful` finishes in-flight requests first.

KEY TAKEAWAY Knowing each startup step tells you where to look when an application won't start.

SPRING BOOT AUTO-CONFIGURATION

Configuration decided by the classpath, properties and existing beans.



BOOT ASKS

- ? Is the library on the classpath?
- ? Is it a web application?
- ? Did you define this bean already?
- ? Does a property turn it on or off?

Example

Add spring-boot-starter-web and you get Spring MVC, Tomcat and Jackson JSON.

IT BACKS OFF Define your own bean and Boot's default steps aside.

KEY TAKEAWAY Auto-configuration is conditional — and it backs away when you define your own configuration.

AUTO-CONFIGURATION IN CODE

Conditions decide; your own bean wins; the report shows every decision.

Inside Spring Boot (simplified)

```
@AutoConfiguration
@ConditionalOnClass(ObjectMapper.class)
public class JacksonConfig {

    @Bean
    @ConditionalOnMissingBean
    ObjectMapper objectMapper() {
        return new ObjectMapper();
    }
}
```

✓ Your bean wins

```
@Configuration
public class JsonConfig {

    @Bean
    ObjectMapper objectMapper() {
        return JsonMapper.builder()
            .findAndAddModules()
            .build();
    }
}
```

See every decision

```
# application.yml
debug: true

# or when starting
java -jar order-service.jar --debug

# the log then shows:
# CONDITIONS EVALUATION REPORT
#   Positive matches: ...
#   Negative matches: ...
```

@ConditionalOnClass

Only if the library is on the classpath.

@ConditionalOnMissingBean

Only if you haven't defined one yourself.

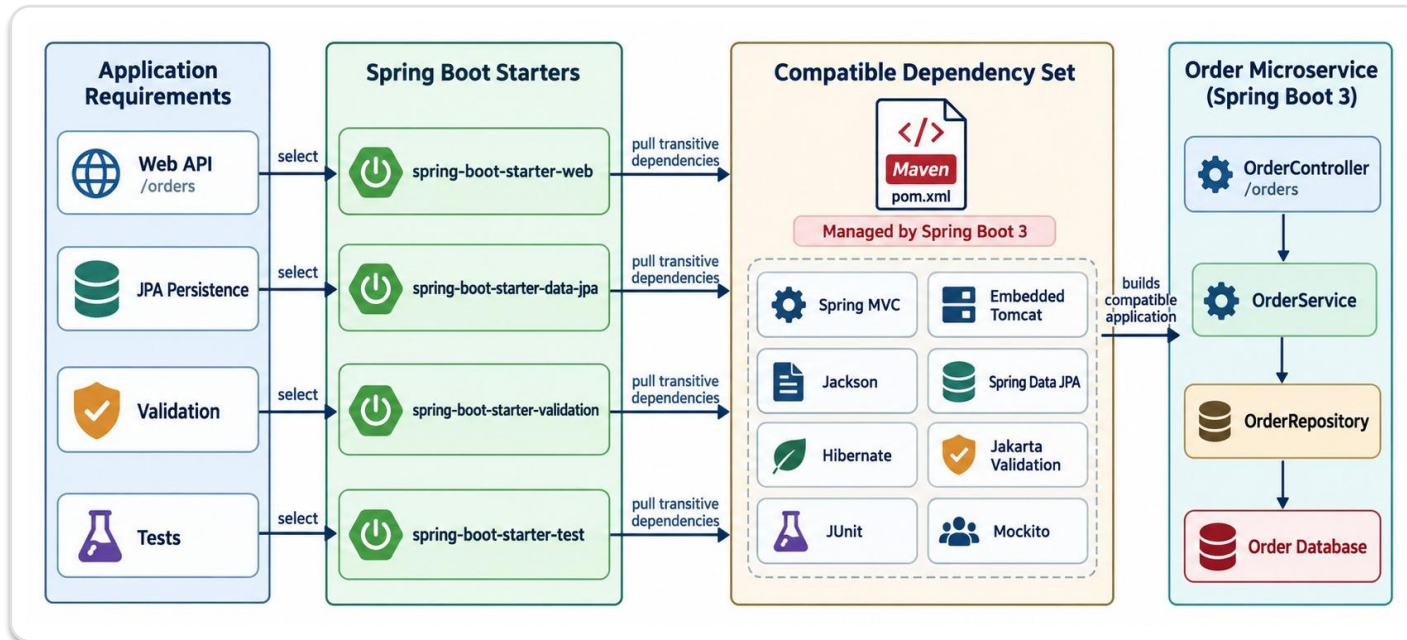
@ConditionalOnProperty

Only if a property turns it on.

KEY TAKEAWAY Auto-configuration is a set of conditions — define your own bean and Boot's default steps aside.

STARTERS AND DEPENDENCY MANAGEMENT

spring-boot-starter- and compatible dependency versions.*



Starter

A curated group of libraries for one capability.

pom.xml

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.5.0</version>
</parent>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <!-- no version: managed by the parent -->
</dependency>
```

CAUTION Override a managed version only when you understand the compatibility risk.

KEY TAKEAWAY Starters bring the right libraries; Boot's dependency management keeps their versions compatible.

EXERCISE: CHOOSE THE STARTERS

Pick exactly what the Order Service needs — and nothing more.

PRACTICE EXERCISE

10 MIN

SCENARIO

Order Service needs:

- a REST API for /orders
- orders saved in PostgreSQL
- request validation
- a health endpoint for OpenShift
- unit and slice tests

A teammate also wants Spring Security, Kafka and Redis “just in case”.

YOUR TASKS

- 1 Pick a starter for each need
- 2 Name one thing each starter auto-configures
- 3 Decide what to do with the “just in case” dependencies

EXPECTED OUTCOME

- ✓ web → MVC, Tomcat, Jackson
- ✓ data-jpa + PostgreSQL driver → DataSource, repositories
- ✓ validation → Bean Validation; actuator → /actuator/health
- ✓ test → JUnit, Mockito, AssertJ
- ✓ Leave the rest out until a requirement needs it

KEY TAKEAWAY Every starter switches on auto-configuration — add one only when the service needs it.

PART 2: BUILDING THE ORDER API

From a running application to working endpoints.

✓ Part 1
What Spring Boot does

Part 2
Build the Order API

Part 3
Portable and running

SO FAR

- ✓ Boot adds starters, auto-configuration and a server to Spring
- ✓ @SpringBootApplication scans from the main class package
- ✓ Startup follows a fixed, readable lifecycle

NEXT QUESTION

How do we turn the Order API contract into working Spring code?

IN THIS PART

- 1 The Order Service case study
- 2 Request flow, mappings and request data
- 3 DTOs, validation and error handling
- 4 Service and repository code
- 5 Dependency injection and beans
- 6 Exercises: refactor and fix the wiring

OUR CASE STUDY: THE ORDER SERVICE

What it does, the files we write, and the API it exposes.

ORDER SERVICE DOES

- ✓ Create an order
- ✓ Retrieve an order
- ✓ Update an order
- ✓ Cancel an order

FROM MODULE 2 It implements the Order capability and the API contract we designed.

FILES WE WILL WRITE

`OrderController`

HTTP in, HTTP out

`CreateOrderRequest · OrderResponse`

The API contract

`OrderService`

Business rules

`OrderRepository`

Data access

`GlobalExceptionHandler`

One error format

POTENTIAL API

POST

`/orders`

GET

`/orders`

GET

`/orders/{id}`

PATCH

`/orders/{id}`

DELETE

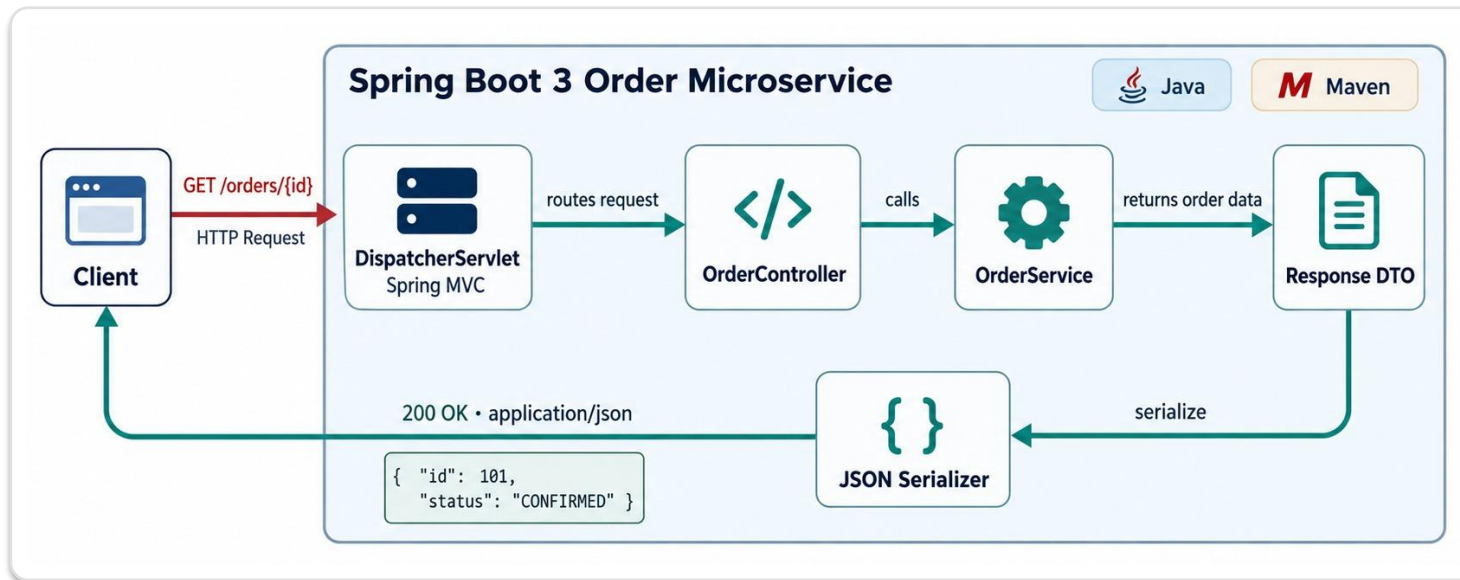
`/orders/{id}`

LAB 1 You build the same layers for the banking Account Service.

KEY TAKEAWAY One Order Service — controller, service and repository — carries us through the module.

REST REQUEST-RESPONSE FLOW

@RestController and @RequestMapping — request in, JSON out.



SPRING MVC DOES FOR YOU

- ✓ Routes the URL to one controller method
- ✓ Converts path, query and body into Java types
- ✓ Serializes the return value to JSON
- ✓ Turns exceptions into error responses

DispatcherServlet

Spring MVC's front controller finds the matching method; Jackson turns the return value into JSON.

THIN CONTROLLER Map HTTP, then delegate to the service.

KEY TAKEAWAY The controller maps the HTTP request, then delegates the business work to the service layer.

MAPPING HTTP OPERATIONS

@GetMapping, @PostMapping, @PutMapping, @PatchMapping, @DeleteMapping.

HTTP	Spring	Example
GET	@GetMapping	/orders/101
POST	@PostMapping	/orders
PUT	@PutMapping	/orders/101
PATCH	@PatchMapping	/orders/101
DELETE	@DeleteMapping	/orders/101

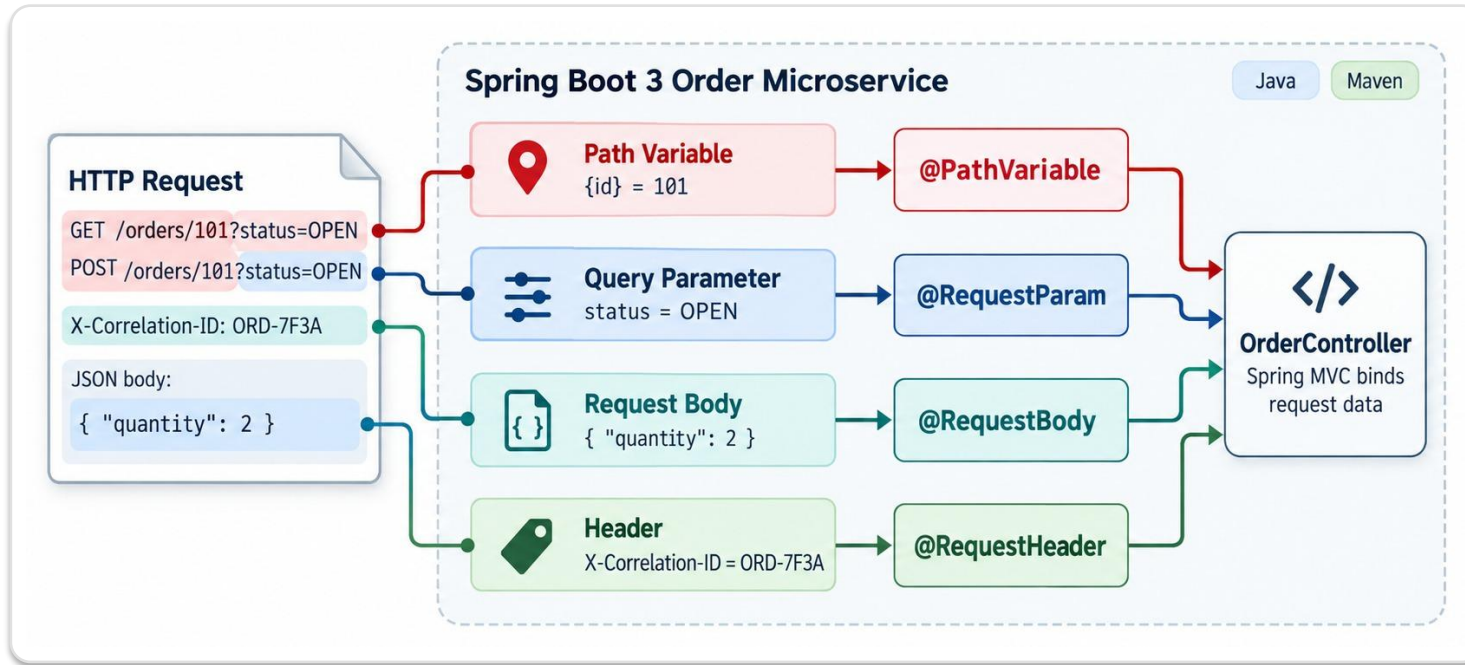
```
@RestController
@RequestMapping("/orders")
class OrderController {
    @GetMapping("/{id}")    // retrieve
    @PostMapping            // create
    @PutMapping("/{id}")    // replace
    @PatchMapping("/{id}")  // partial
    @DeleteMapping("/{id}") // delete
}
```

DESIGN CAME FIRST Spring annotations implement the HTTP contract — they don't decide what a good contract is.

KEY TAKEAWAY Each mapping annotation implements the REST contract we designed in Module 2.

WORKING WITH REQUEST DATA

@PathVariable, @RequestParam, @RequestBody and @RequestHeader.



One method, three sources

```
@GetMapping("/{id}")
OrderResponse get(
    @PathVariable Long id,
    @RequestParam(required = false)
        String status,
    @RequestHeader("X-Correlation-Id")
        String correlationId) { ... }
```

Type conversion

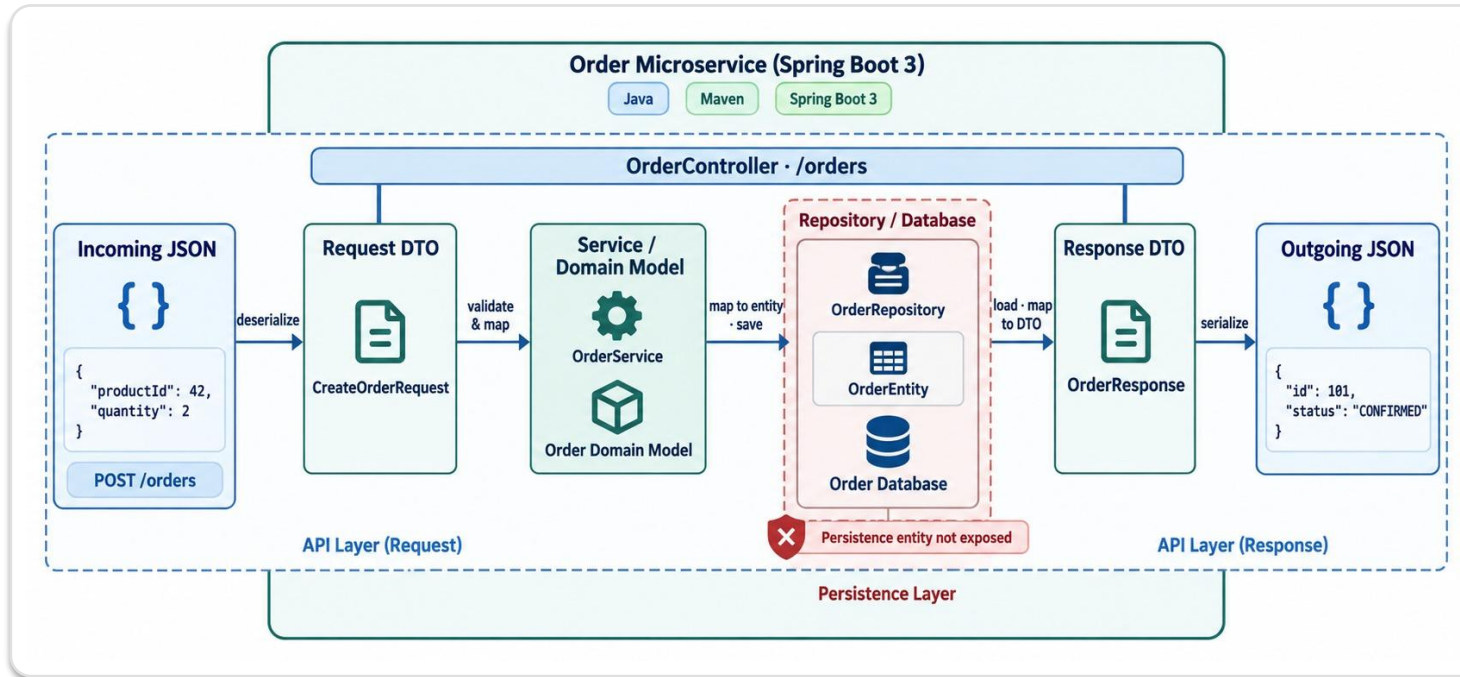
Spring turns "101" into a Long. A value it can't convert returns 400.

OPTIONAL VALUES Use `required = false` or a `defaultValue`.

KEY TAKEAWAY Spring maps each part of the HTTP request into Java method parameters — including JSON bodies.

DTOS AND API RESPONSES

Request and response DTOs keep the entity inside the service.



NEVER LEAVES THE SERVICE

- ✗ internalProcessingCode
- ✗ databaseVersion
- ✗ Audit fields

✓ Map entity → DTO in one place

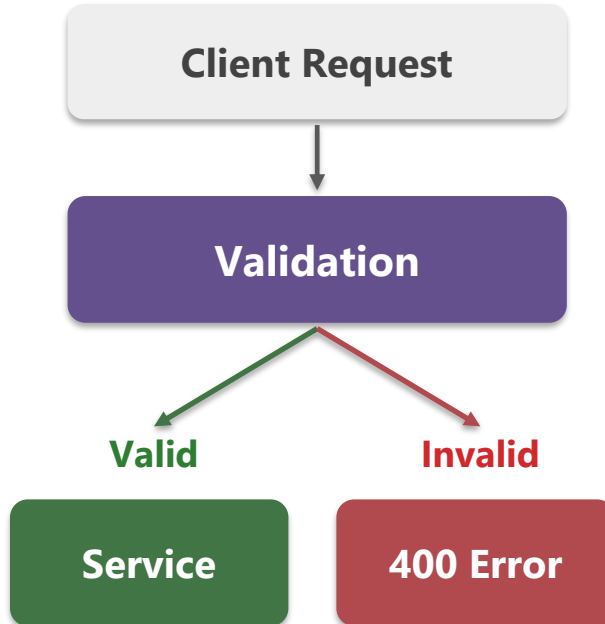
```
public record OrderResponse(  
    Long id, String status,  
    BigDecimal total) {  
  
    static OrderResponse from(Order o) {  
        return new OrderResponse(o.getId(),  
            o.getStatus().name(), o.getTotal());  
    }  
}
```

MORE CONTROL `ResponseEntity<T>` sets status, headers and body.

KEY TAKEAWAY Don't expose entities — DTOs give clients exactly the contract they need.

VALIDATION AND ERROR HANDLING

Bean Validation, @Valid, consistent errors and @RestControllerAdvice.



GOOD ERROR RESPONSE

**errorCode · message ·
timestamp · path · traceId**

CONSTRAIN THE REQUEST DTO

```
public record CreateOrderRequest(  
    @NotNull Long customerId,  
    @NotEmpty List<OrderItemRequest> items) {  
}
```

TRIGGER VALIDATION IN THE CONTROLLER

```
@PostMapping  
public ResponseEntity<OrderResponse> create(  
    @Valid @RequestBody  
    CreateOrderRequest request) { ... }
```

CENTRALIZE @RestControllerAdvice +
@ExceptionHandler give consistent API errors.

KEY TOOLS

@Valid

@NotNull

@NotBlank

@Size

@Positive

@RestControllerAdvice

@ExceptionHandler

KEY TAKEAWAY Never trust incoming requests — validate them, and return consistent errors from one place.

CENTRALIZED ERROR HANDLING

One handler turns exceptions into the error format clients rely on.

Throw a meaningful exception

```
public class OrderNotFoundException
    extends RuntimeException {

    public OrderNotFoundException(
        Long id) {
        super("Order " + id
            + " was not found");
    }
}
```

One place

No try/catch copied into every controller.

✓ Handle it in one place

```
@RestControllerAdvice
class GlobalExceptionHandler {

    @ExceptionHandler(
        OrderNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    ErrorResponse notFound(
        OrderNotFoundException ex,
        HttpServletRequest req) {
        return ErrorResponse.of(
            "ORDER_NOT_FOUND",
            ex.getMessage(),
            req.getRequestURI());
    }
}
```

Right status

Validation 400, not found 404, conflict 409.

What the client receives

```
HTTP/1.1 404 Not Found

{
  "code": "ORDER_NOT_FOUND",
  "message": "Order 999 was not found",
  "timestamp": "2026-09-14T10:15:30Z",
  "path": "/orders/999"
}
```

Safe messages

Log details inside; never return stack traces.

KEY TAKEAWAY Throw meaningful exceptions and map them once — every endpoint then returns the same error shape.

SERVICE AND REPOSITORY LAYERS IN CODE

Rules and transactions in the service; persistence in the repository.

OrderService.java

```
@Service
public class OrderService {

    private final OrderRepository orders; // injected

    @Transactional
    public OrderResponse cancel(Long id) {
        Order order = orders.findById(id)
            .orElseThrow(() -> new OrderNotFoundException(id));
        order.cancel(); // rule: only CREATED orders
        return OrderResponse.from(order);
    }
}
```

Service

Owns business rules and the transaction boundary.

✓ OrderRepository.java

```
public interface OrderRepository
    extends JpaRepository<Order, Long> {

    List<Order> findByCustomerId(String customerId);

    long countByCustomerIdAndStatus(
        String customerId, OrderStatus status);
}

// Spring Data writes the implementation:
// save, findById, findAll, deleteById, count
```

Repository

Persistence only. Method names become queries.

NEVER Call a repository straight from a controller.

KEY TAKEAWAY Services decide and repositories store — keep business rules out of both controllers and repositories.

EXERCISE: REFACTOR A FAT CONTROLLER

Move each responsibility to the layer where it belongs.

PRACTICE EXERCISE

12 MIN

SCENARIO

A teammate wrote this controller method:

```
@PostMapping("/orders")
Order create(@RequestBody Order o) {
    if (o.getItems().isEmpty())
        return null;
    int open = jdbc.queryForObject(
        "select count(*) from orders"
        + " where customer_id = ?",
        Integer.class, o.getCustomerId());
    if (open > 5)
        throw new RuntimeException("limit");
    o.setStatus("CREATED");
    System.out.println("Saved " + o);
    return orderRepository.save(o);
}
```

YOUR TASKS

- 1 Label each line: controller, validation, service, repository or logging
- 2 Rewrite the signature with a request DTO, @Valid and a 201 response
- 3 Name the exception and status code for each failure

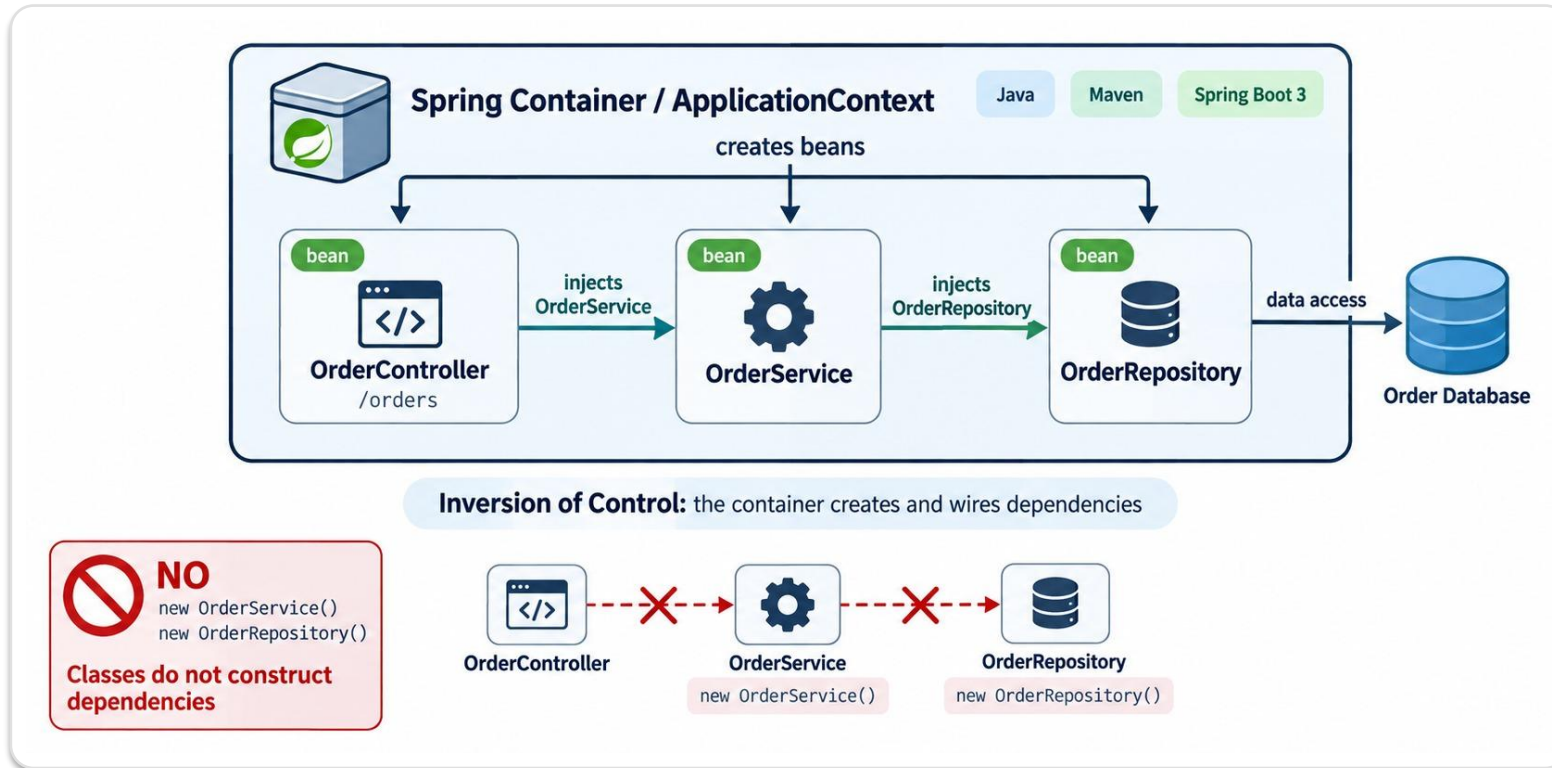
EXPECTED OUTCOME

- ✓ Empty items → @NotEmpty on the DTO → 400
- ✓ Open-order limit → service rule → 409
- ✓ Count query → repository method
- ✓ Status CREATED → service or domain
- ✓ println → log.info with orderId only

KEY TAKEAWAY A thin controller, a rule-owning service and a query-only repository are easier to test and change.

DEPENDENCY INJECTION IN SPRING BOOT

The container builds the objects; classes only declare what they need.



INJECTION STYLES

- ✓ Constructor — preferred
- ✓ Setter — optional dependencies
- ✗ Field @Autowired — avoid
- ✓ **Unit test without Spring**

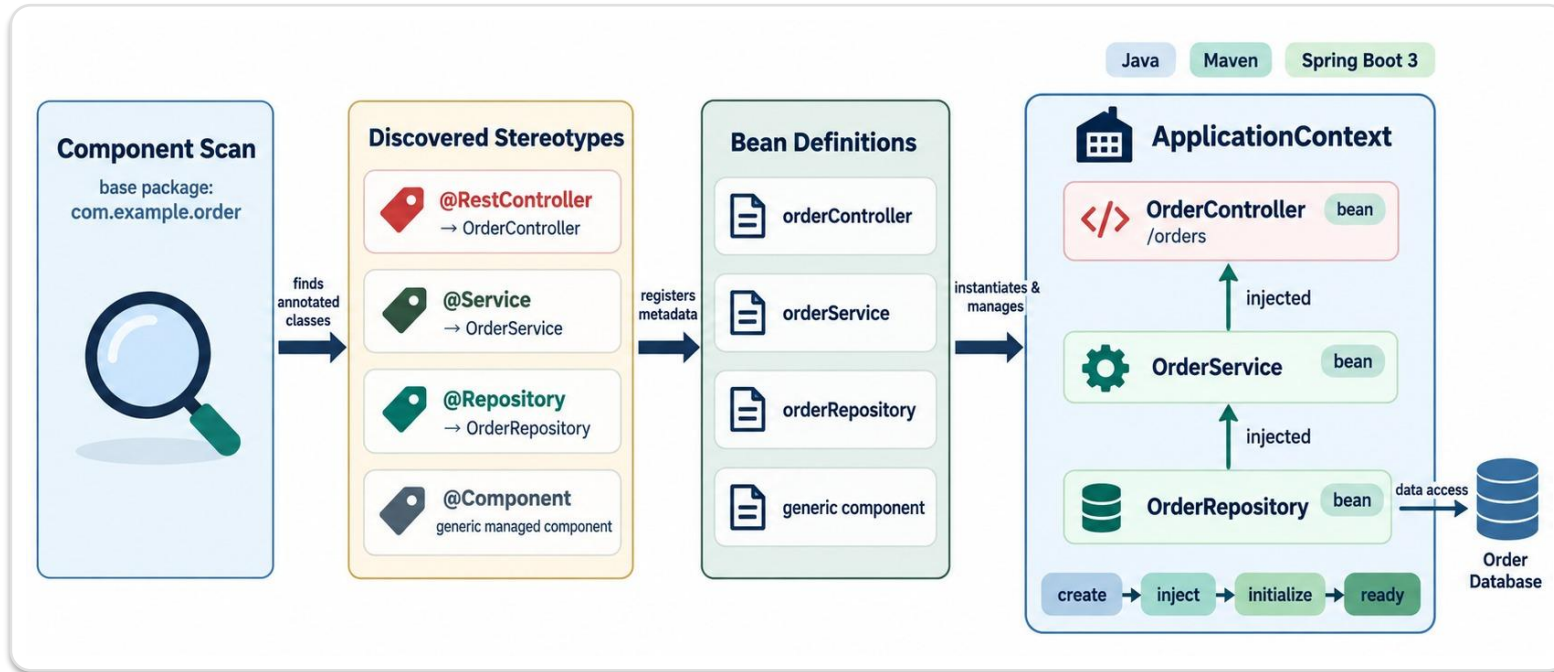
```
var repo = mock(OrderRepository.class);  
var service = new OrderService(repo);
```

ONE CONSTRUCTOR? No @Autowired needed.

KEY TAKEAWAY A class declares what it needs, not how to build it — use constructor injection by default.

COMPONENT SCANNING AND BEAN MANAGEMENT

Stereotype annotations, @Bean methods and the ApplicationContext.



A bean

An object the container creates and manages. One shared instance by default (singleton).

Declare a bean explicitly

```
@Configuration
class ClientConfig {
    @Bean
    RestClient paymentClient(
        RestClient.Builder b) {
        return b.baseUrl("http://payment")
            .build();
    }
}
```

USE @BEAN FOR classes you can't annotate, like library clients.

KEY TAKEAWAY Stereotype annotations and @Bean methods put objects under the Spring container's management.

EXERCISE: WHY WON'T IT WIRE?

Read the startup error and fix the wiring.

PRACTICE EXERCISE

10 MIN

SCENARIO

Startup fails: "Parameter 0 of constructor in OrderController required a bean of type OrderService that could not be found."

```
package com.example.order;
@SpringBootApplication
public class OrderApplication { }

package com.example.services;
public class OrderService { }

package com.example.order.web;
@RestController
public class OrderController {
    @Autowired
    private OrderService service;
}
```

YOUR TASKS

- 1 Find two reasons OrderService is not a bean
- 2 Fix them, and replace the field injection
- 3 Say which startup step failed

EXPECTED OUTCOME

- ✓ No @Service annotation
- ✓ com.example.services is outside the scanned package
- ✓ Move it under com.example.order and add @Service
- ✓ Use a final field and a constructor
- ✓ Failed while creating beans — before Tomcat starts

KEY TAKEAWAY A missing bean is almost always a missing annotation or a class outside the scanned package.

PART 3: PORTABLE AND RUNNING

A working API is not finished until it runs anywhere and tells you how it's doing.

✓ Part 1
What Spring Boot does

✓ Part 2
Build the Order API

Part 3
Portable and running

SO FAR

- ✓ Controllers map HTTP and stay thin
- ✓ DTOs, validation and one handler form the contract
- ✓ Spring creates and injects every bean

NEXT QUESTION

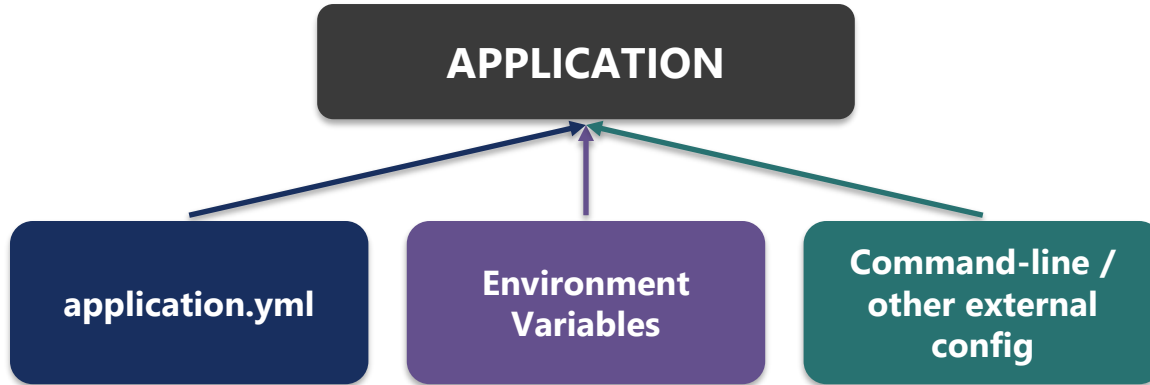
How does one JAR run in dev, test and production — and how do we know it's healthy?

IN THIS PART

- 1 Externalized and type-safe configuration
- 2 Profiles and environments
- 3 Exercise: which value wins?
- 4 From source to a running service
- 5 Troubleshooting startup, and logging
- 6 The complete picture

EXTERNALIZED CONFIGURATION

application.yml, environment variables and configuration precedence.



EXAMPLE APPLICATION.YML

```
spring:
  application:
    name: order-service

server:
  port: 8080
```

AVOID — HARD-CODED IN JAVA

```
String databaseUrl =
    "jdbc:postgresql://prod-server/orders";
```

PREFER — EXTERNALIZED VALUES

```
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USER}
    password: ${DB_PASSWORD}
```

Same artifact, different environments

→ *Twelve-Factor "Config" from Module 1*

KEY TAKEAWAY Keep configuration out of Java code so the same artifact runs in every environment.

TYPE-SAFE CONFIGURATION

@ConfigurationProperties binds a whole group of settings to one Java type.

application.yml

```
orders:
  max-items-per-order: 50
  payment:
    base-url: ${PAYMENT_URL}
    timeout: 2s
```

✓ OrderProperties.java

```
@Validated
@ConfigurationProperties("orders")
public record OrderProperties(
    @Min(1) int maxItemsPerOrder,
    @Valid Payment payment) {

    public record Payment(
        @NotBlank String baseUrl,
        Duration timeout) {}
}
```

Enable it, then inject it

```
@SpringBootApplication
@ConfigurationPropertiesScan
public class OrderApplication { }

@Service
class OrderService {
    OrderService(OrderProperties p) {
        this.maxItems =
            p.maxItemsPerOrder();
    }
}
```

vs. @Value

One typed object instead of scattered strings.

Fail fast

A bad value stops startup and names the field.

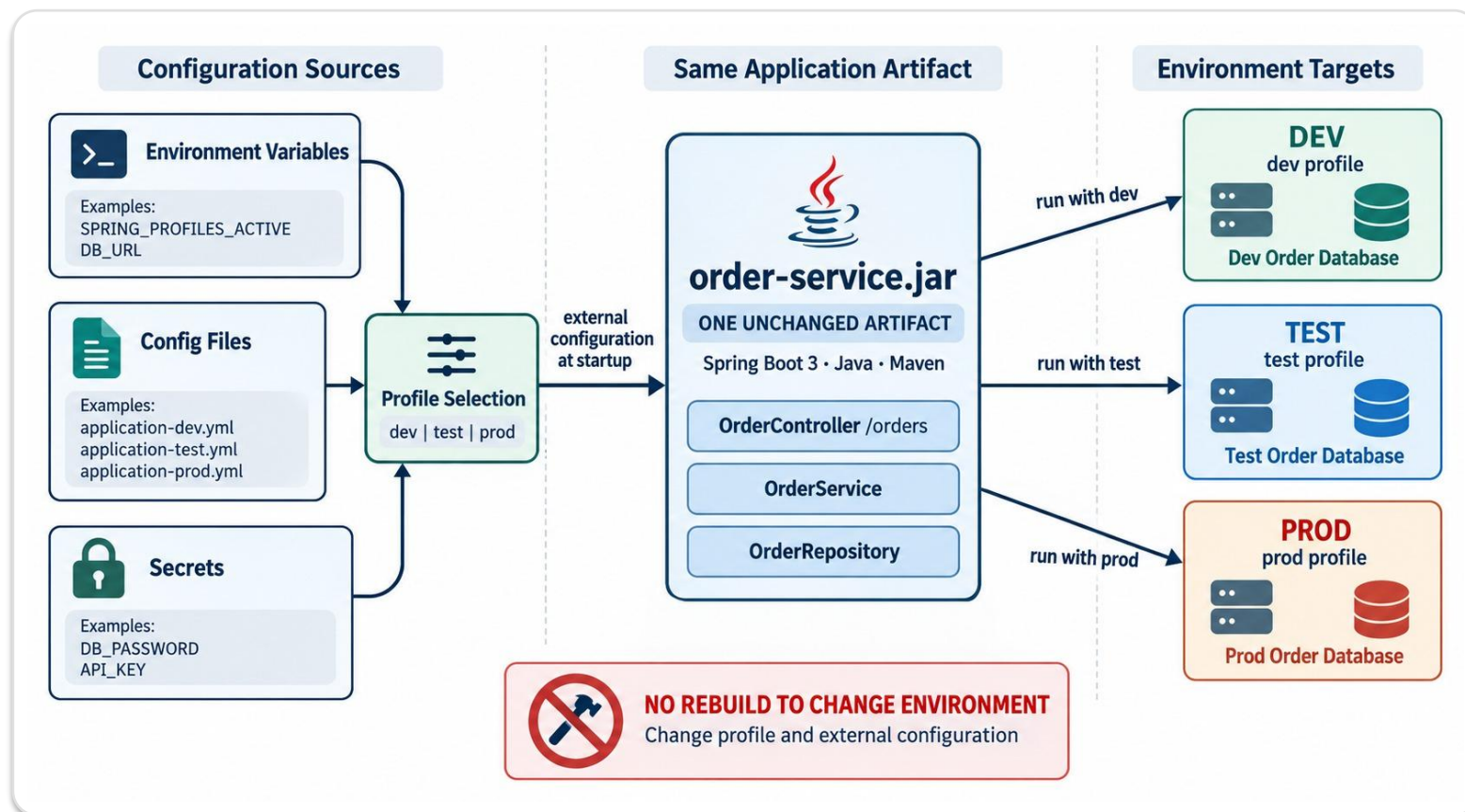
From the environment

ORDERS_MAXITEMSPERORDER=20 overrides the YAML.

KEY TAKEAWAY Group related settings in a validated properties type — misconfiguration then fails at startup, not in production.

PROFILES: ONE ARTIFACT, MANY ENVIRONMENTS

dev, test and prod configuration — with secrets supplied by the platform.



Base file plus overrides

```
application.yml      # shared
application-dev.yml
application-prod.yml
```

Choose the profile at runtime

```
java -jar order-service.jar \
  --spring.profiles.active=prod

# on OpenShift
SPRING_PROFILES_ACTIVE=prod
```

SECRETS Never in YAML. The platform injects `DB_PASSWORD` at runtime.

KEY TAKEAWAY Build once — profiles, environment variables and secrets adapt the same JAR to each environment.

EXERCISE: WHICH VALUE WINS?

Predict the configuration Spring Boot actually uses.

PRACTICE EXERCISE

10 MIN

SCENARIO

Order Service is started three ways:

A: no profile

B: prod profile

C: prod profile, env SERVER_PORT=9090 and
command-line server.port=7070

```
# application.yml
server.port: 8080
orders.max-items-per-order: 50
```

```
# application-prod.yml
server.port: 8081
orders.max-items-per-order: 20
```

YOUR TASKS

- 1 Give the port and item limit for A, B and C
- 2 Rank the sources from highest to lowest priority
- 3 Decide where the database password should come from

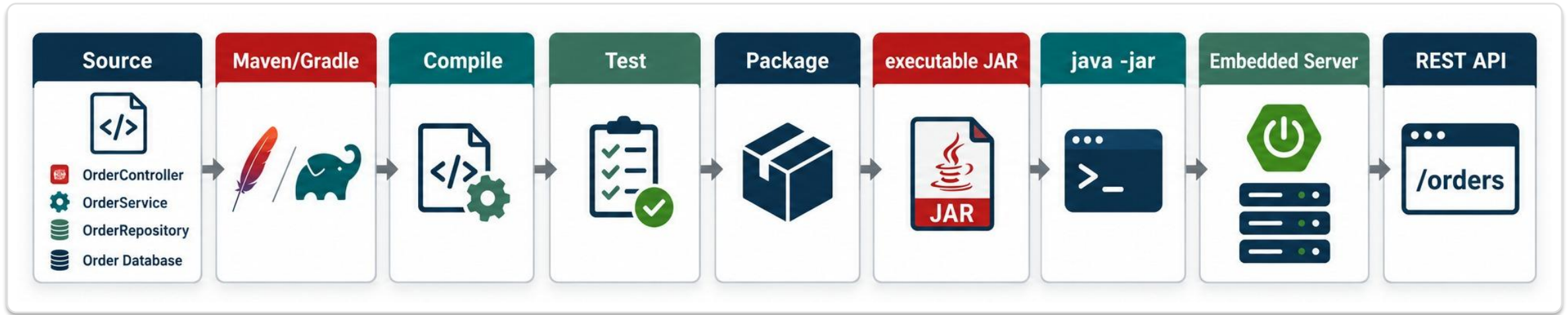
EXPECTED OUTCOME

- ✓ A: 8080 and 50
- ✓ B: 8081 and 20
- ✓ C: 7070 and 20 — command line wins
- ✓ Command line > env vars > profile file > application.yml
- ✓ Password: a platform secret, never a file

KEY TAKEAWAY Command line beats environment variables, which beat profile files, which beat application.yml.

FROM SOURCE TO A RUNNING SERVICE

Build one executable JAR, start it, call it.



1 Build

```
./mvnw clean package  
# compiles, runs tests, packages
```

2 Run

```
java -jar target/order-service.jar  
# dev loop: ./mvnw spring-boot:run
```

✓ 3 Call

```
curl localhost:8080/orders/101  
# or Postman, IDE HTTP client
```

KEY TAKEAWAY A build tool compiles, tests and packages one executable JAR — java -jar starts the server and the API.

WHEN THE SERVICE WON'T START

Match the log message to the startup step, then check the likely cause.

Symptom in the log	Startup step	Check first
Port 8080 was already in use	Embedded server	Stop the other process or change server.port
No qualifying bean of type OrderService	Create beans	@Service present? Inside the scanned package?
Could not resolve placeholder 'DB_URL'	Prepare environment	Variable set? Right profile active?
Connection refused: localhost:5432	Create beans (DataSource)	Database running? URL correct?
Binding to target OrderProperties failed	Prepare environment	Is the value valid for its type?

✓ **Is it healthy?**

```
curl localhost:8080/actuator/health
{"status": "UP"}
```

Read from the top
The first error is the cause. Later errors are usually side effects.

WRONG PROFILE? Look for “The following profiles are active” near the top.

KEY TAKEAWAY Find the failing startup step in the log — it points straight at the cause.

LOGGING THAT HELPS IN PRODUCTION

SLF4J with Logback: clear messages, useful IDs, no sensitive data.

✗ Avoid

```
System.out.println("Order: " + order);

log.info("Creating order " + request);

log.info("Card {} charged", cardNumber);

log.error("Failed");
```

✓ Prefer

```
private static final Logger log =
    LoggerFactory.getLogger(OrderService.class);

log.info("Order created orderId={} status={}",
    order.getId(), order.getStatus());

log.warn("Order {} rejected: item limit", id);

log.error("Payment call failed orderId={}", id, ex);
```

Parameterized

Placeholders with IDs — never whole objects or bodies.

Never log

Card numbers, emails, tokens or request bodies.

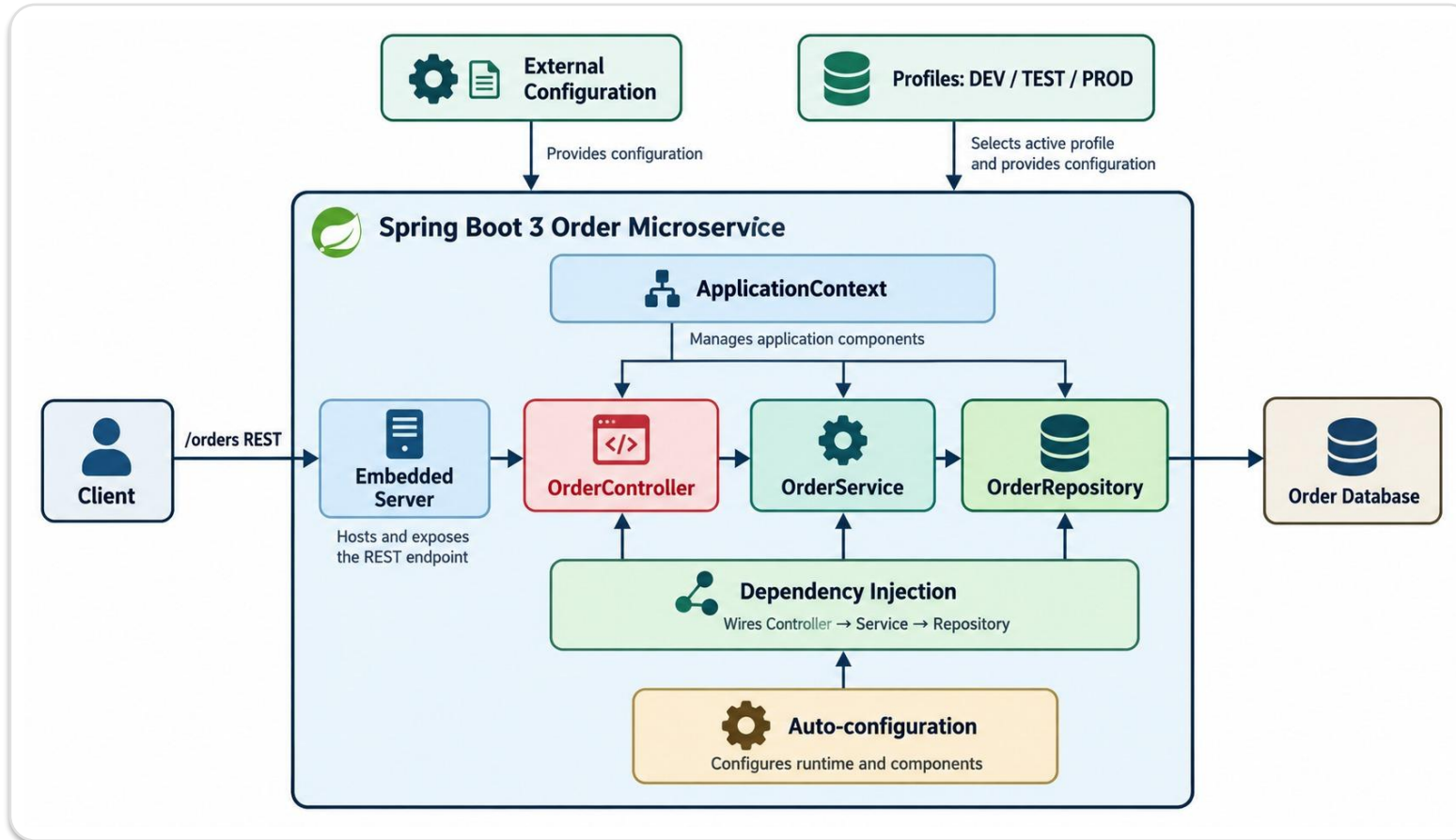
Levels by profile

DEBUG locally; INFO and WARN in production.

KEY TAKEAWAY Log events with IDs at the right level — and never log sensitive customer data.

SPRING BOOT MICROSERVICE — END TO END

The layers, plus the runtime mechanisms that make them work.



THE FULL JOURNEY

- ✓ App starts; configuration loads
- ✓ Beans are created and injected
- ✓ Tomcat starts on the port
- ✓ POST /orders reaches the controller
- ✓ Service applies rules; repository saves
- ✓ JSON response returns

COMPARE The layered view showed only the layers. This adds the runtime around them.

KEY TAKEAWAY Spring Boot turns our microservice design into a runnable application.

SPRING BOOT MISCONCEPTIONS AND ANTI-PATTERNS

Beliefs and habits that make Spring Boot services hard to run and change.

MISCONCEPTION

REALITY

ANTI-PATTERNS TO AVOID

Spring Boot is magic

It follows conditions you can inspect with -
-debug

Boot replaced the Spring Framework

It builds on Spring and configures it for
you

The defaults are production-ready

Review pools, timeouts, logging and
security

@Autowired fields are fine

Constructor injection is explicit and
testable

Profiles are the place for secrets

Secrets come from the platform, not YAML

- ✗ Business logic in controllers
- ✗ Returning JPA entities from APIs
- ✗ One giant service class
- ✗ Hard-coded URLs and passwords
- ✗ System.out instead of a logger
- ✗ Adding starters "just in case"

REMEMBER Spring Boot speeds up good design — it doesn't create it.

KEY TAKEAWAY Treat Spring Boot as transparent rules you control — and keep each layer's job clean.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1

What does Spring Boot add to traditional Spring setup?

2

Which layer holds business rules?

3

Why should controllers stay thin?

4

What is the repository layer for?

5

Why prefer constructor injection over field injection?

6

application.yml vs. application-prod.yml?

7

Why keep passwords out of config files?

8

Which annotation triggers request-body validation?

9

Why centralize exception handling?

10

Why package the service as an executable JAR?

EXERCISE 1.3 — SCAFFOLD ACCOUNT ENDPOINTS AND VALIDATION

Module 3 checkpoint C: controllers, DTOs and validation rules.

OFFICIAL CHECKPOINT C

15 MIN

curriculum/day-01/exercises/exercise-1.3-scaffold-account.md

SCENARIO

Lab 1 builds the banking Account Service (starter on port 8081). Before coding, map this module to the starter project.

Scope: Account, Transaction and Risk Assessment only — no extra services.

YOUR TASKS

- 1 Open labs/day-01/lab1/starter; find the controller, service, DTO and validation TODOs
- 2 Name the layers: Controller → Service → Repository → PostgreSQL (Flyway, not Hibernate DDL)
- 3 List the rules: customerId CUST- plus four digits, nickname length, legal status change; accountId is generated
- 4 Confirm logs use accountId and status only — no request bodies or PII

EXPECTED OUTCOME

- ✓ You can point at every starter file Lab 1 completes
- ✓ Layer names match the lab
- ✓ Validation rules are ready to code
- ✓ Don't skip ahead into Lab 2

KEY TAKEAWAY You can point at the starter files Lab 1 will complete — without skipping ahead to Lab 2.

MODULE 3 SUMMARY AND WHAT'S NEXT

From a Spring Boot project to a running, configurable service.

YOU CAN NOW

- ✓ Explain what Spring Boot adds on top of Spring
- ✓ Follow startup, auto-configuration and component scanning
- ✓ Build a thin controller with DTOs, validation and errors
- ✓ Wire services and repositories with constructor injection
- ✓ Configure, build and run one JAR in every environment

NEXT: MODULE 4 — MICROSERVICES INTEGRATION

- 1 How do services talk without waiting on each other?
- 2 When do we use queues, pub/sub and events?
- 3 How do we make message consumers reliable?

BRING ALONG Your Lab 1 Account Service, running on port 8081.

KEY TAKEAWAY Module 4 connects services like this one through messaging and events.